AI Agent Forensics & Audit Platform:

Evidence Capture, Tamper-Evident Logging and Post-Incident Reconstruction for LLM-Based Agents

Project Synopsis Submitted

to

MANIPAL ACADEMY OF HIGHER EDUCATION

For Partial Fulfillment of the Requirement for the

Award of the Degree

Of

Bachelor of Technology

in

Data Science & Engineering

by

Pandiri Jaswanth Kumar – Reg. No. 230968014

Kosuru Venkata Sai Aditya – Reg. No. 230968316

Under the guidance of

Dr. Sindhura D N
Assistant Professor
School of Computer Engineering
Manipal Institute of Technology
MAHE, Manipal, Karnataka, India



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
A Constituent Unit of MAHE, Manipal

October 2026

Abstract

LLM-based agents plan tasks, call external tools and act on live data, but the records they leave behind are rarely fit to serve as digital evidence. This project proposes a platform that captures every step of an agent session (prompts, retrieved context, tool calls, outputs and configuration) as structured events, stores them in a hash-chained and signed log so that tampering can be detected, and provides tools to rebuild an incident timeline and flag suspicious behaviour such as prompt injection and out-of-policy tool use. The platform will be tested on public agent-attack benchmarks and on traces generated from a purpose-built test agent, and evaluated for detection accuracy, evidence completeness and overhead.

Problem Statement and Motivation

There is no common, practical way to collect forensically sound evidence from LLM agent executions. Teams usually depend on application logs or observability dashboards meant for debugging, which can be edited without a trace, scatter prompts, context and tool results across different places, and cannot link an action to the exact input and configuration behind it. Nothing records who collected the data or whether it was verified, so an organisation facing an incident often cannot show what happened or whether its records are intact.

The need is growing. ISO/IEC 27037 sets out how digital evidence should be identified, collected and preserved [1]; Article 12 of the EU AI Act requires automatic event logging for high-risk AI systems [2]; and the OWASP list of LLM application risks places prompt injection and excessive agency among the top concerns [3]. Existing tooling does not yet connect these expectations to agent systems.

Objectives

1. Define an evidence model for agent sessions covering prompts, responses, retrieved context, tool calls and results, and the configuration in force.
2. Build a capture layer that records this evidence automatically with little change to the agent's own code.
3. Provide tamper-evident storage using hash chaining and signed checkpoints, with an independent verifier.
4. Maintain chain-of-custody records for collection and verification.
5. Reconstruct session timelines and detect suspicious patterns such as injected instructions and unauthorised tool calls.
6. Evaluate detection accuracy, evidence completeness and runtime overhead on simulated attacks.

Literature Review

Forensic foundations. NIST SP 800-86 explains how forensic techniques fit into incident response, from collection through examination and reporting [4]. These guidelines assume stable artefacts such as disk images and logs, which an agent session does not naturally provide. **Tamper-evident logging.** Schneier and Kelsey showed how cryptographic chaining can make audit logs trustworthy even after a machine is compromised [5], and Crosby and Wallach designed efficient history-tree structures that allow logs to be checked by untrusting parties [6]. This work supplies the integrity mechanism reused here.

Agents and their attack surface. ReAct showed how models can interleave reasoning with tool actions, the pattern now common in agent frameworks [7]. Greshake et al. demonstrated that instructions hidden in retrieved content can hijack LLM-integrated applications (indirect prompt injection) [8]. AgentDojo [9] and InjecAgent [10] then provided benchmarks for measuring such attacks against tool-using agents, but they focus on attack success and defences rather than on evidence collection after an incident. **Auditing.** Raji et al. proposed an internal audit framework for algorithmic systems that stresses documentation and traceability across the development lifecycle [11]; it does not address runtime forensic evidence from autonomous agents. This project fills that gap by joining tamper-evident logging with agent-specific analysis.

Methodology

- **Threat modelling.** Fix the incident classes in scope (direct and indirect prompt injection, data exfiltration through tools, tool privilege misuse) and map each to the evidence needed.
- **Evidence capture.** Hook into the agent framework's callbacks and store each step as a JSON event with session ID, sequence number, timestamp, actor and payload; record model identifier, parameters and system prompt at session start.
- **Integrity layer.** Chain events by including the previous event's SHA-256 hash in each record, sign the chain head at regular intervals, and provide a verifier that reports the first point where the chain breaks.
- **Forensic analysis.** Build a timeline viewer and rule-based detectors for policy violations, instructions in retrieved content that influence later actions, and sensitive data in outputs. Any LLM-assisted triage must link findings back to raw records.
- **Experiments and reporting.** Run benign and attack scenarios, deliberately tamper with stored logs, and measure detection rate, false positives, completeness and latency against an uninstrumented baseline. Generate an audit report per incident.

Tools, Technologies and Datasets

Tools and technologies (proposed): Python; LangChain or LangGraph for the test agent; an open-source or hosted LLM; SHA-256 hashing and Ed25519 signatures through standard cryptography libraries; SQLite or a similar append-only store for events; FastAPI or Streamlit for the timeline viewer and report generation.

Datasets: (i) attack scenarios and tool-use tasks from the AgentDojo [9] and InjecAgent [10] benchmarks; (ii) benign and attack traces generated by the project's own test agent, which serve as ground truth for evidence completeness; (iii) a set of deliberately tampered log files (edits, deletions, reordering) for testing the verifier.

Expected Outcome

- A working prototype with capture, tamper-evident storage, verification and analysis modules.
- A reusable evidence schema for LLM agent sessions.
- Measured results on tamper detection and on the simulated attack classes, including overhead.
- Audit reports in which each finding is tied to verifiable raw records.
- A clear statement of limits: model internals cannot be recovered and non-deterministic output prevents exact replay, so stored evidence, not a re-run, is the record.

References

[1] ISO/IEC 27037:2012, "Information technology – Security techniques – Guidelines for identification, collection, acquisition and preservation of digital evidence," Int. Org. Standardization, Geneva, Switzerland, 2012.

[2] European Union, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," *Off. J. Eur. Union*, 2024, Art. 12.

[3] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications." [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[4] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to integrating forensic techniques into incident response," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Spec. Publ. 800-86, 2006.

[5] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, May 1999.

[6] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. 18th USENIX Security Symp.*, Montreal, QC, Canada, 2009.

[7] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[8] K. Greshake *et al.*, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop Artif. Intell. Secur. (AISec)*, 2023.

[9] E. Debenedetti *et al.*, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets Benchmarks Track*, 2024.

[10] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings Assoc. Comput. Linguistics: ACL*, 2024.

[11] I. D. Raji *et al.*, “Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing,” in *Proc. Conf. Fairness, Accountability, Transparency (FAT*)*, 2020, pp. 33–44.

